PROJECT GUIDE / 01

GitPageDocs Project Guide

Documentation that travels with your repository.

A practical guide to configuring, reading and publishing multilingual documentation with GitPageDocs. This PDF is also the working example for the native PDF container introduced in version 0.0.9.

What the project provides

GitPageDocs separates documentation data from its viewer. The CLI creates configuration and content; the Next.js application renders the documentation locally, on GitHub Pages or through the official repository viewer. A shared tools package supports the CLI and the Model Context Protocol (MCP) server.

Read	Markdown, HTML, video, audio, source browsing and PDF containers in one documentation interface.
Navigate	Localized menus, recursive categories, breadcrumbs, keyboard navigation and document versions.
Customize	Layouts and themes, text or icon controls, route presentation, focus actions and native narration.
Publish	Version-controlled documentation, a static GitHub Pages build, and separate CLI, tools and MCP npm packages.

Inside this guide

02 / Start	Installation, the generated files and the repository architecture.
03 / Structure	Routes, nested menus, breadcrumbs and configurable icons.
04 / PDF example	Configure this document as a PDF route and menu entry.
05 / Reading	Focus mode, Markdown actions, narration and media coordination.
06 / Operate	CLI workflows, deployment and package publishing.
07 / Connect	MCP installation, tools, resources and permissions.
08 / Maintain	Validation, troubleshooting and reference documentation.

Edition: 0.0.10. The English, Portuguese and Spanish Markdown manuals provide the detailed, maintained reference. This PDF is a portable overview and configuration example.

Get started

Use Node.js 20 or newer. Install the CLI in the repository whose documentation you want to create. Git is needed when committing or publishing; GitHub CLI is used by automated Pages setup.

```
npm install --save-dev @gitpagedocs/cli
npx gitpagedocs --no-interactive --owner YOUR_USER --repo YOUR_REPOSITORY
```

The generator writes a documentation scaffold. It does not install a complete Next.js application into the target repository. Review the diff after regenerating, especially when you maintain customized Markdown or configuration.

The file contract

Path	Purpose
gitpagedocs/config.json	Site settings, feature switches, icons and registered versions.
gitpagedocs/langs/	Localized interface strings: en.json, pt.json and es.json.
Version config.json	Under gitpagedocs/docs/versions/0.0.10/: routes, menus, shared route defaults and content order.
Version language folders	The en, pt and es folders beside the version config contain localized Markdown. Filenames remain English in every language.
gitpagelayouts/	Optional layout configuration, generated with --layoutconfig.

Develop the viewer

```
git clone https://github.com/Vidigal-code/git-page-docs.git
cd git-page-docs
corepack enable
pnpm install --frozen-lockfile
pnpm dev
```

The repository pins pnpm in packageManager. The default development command enables repository search. Use pnpm dev:e2e for local documentation mode. Set PORT in the environment when the default port is occupied.

Architecture

frontend/ uses the Next.js App Router, React and TypeScript, organized into app, processes, page slices, widgets, features, entities and shared modules. **cli/** provides commands and generators. **tools/** contains shared configuration, filesystem and documentation logic. **mcp/** exposes the shared capabilities through stdio.

Structure and presentation

A documentation version has separate route and menu collections for each content type: md, source-viewer, html, video, audio and pdf. Entries sharing a page ID can contribute multiple content blocks. hierarchyPage determines the block order; hierarchyMenu determines the menu group order.

Menus can contain more menus

Use submenus recursively to organize categories and destinations. A category can have an empty path-click. Localized menu entries contain title and path-click; use the actual localized file path for a Markdown destination. PDF and media routes support page:<id> destinations. There is no configured nesting-depth limit.

```
"menus-header-md": [{
  "id": 101,
  "en": {
    "title": "Guides", "path-click": "",
    "submenus": [{
      "title": "Setup",
      "path-click": "gitpagedocs/docs/versions/0.0.10/en/getting-started.md"
    }]
  }
}]
```

Set site.nestedMenusEnabled to false for a flat presentation. Navigation and access checks still use the same destinations. Add pt and es entries to localize the menu.

Breadcrumbs above Markdown

site.RouteGuide enables the trail by default. A version can override it in routeDefaults.RouteGuide; a Markdown route can override that value with RouteGuide. The most specific configured value wins. RouteguideBrand is a separate setting for the outline of Markdown headings.

Text controls or React Icons

```
"icons": {
  "QuickNavigation": { "reactIcon": true, "tag": "FiSearch" },
  "LanguageSelector": { "reactIcon": true, "tag": "FiGlobe" },
  "ThemeSelector": { "reactIcon": true, "tag": "BsSunFill" },
  "FocusMode": { "reactIcon": true, "tag": "MdFullscreen" },
  "PdfDownload": { "reactIcon": true, "tag": "FiDownload" },
  "PdfOpenInNewTab": { "reactIcon": true, "tag": "FiExternalLink" },
  "DocumentPrevious": { "reactIcon": true, "tag": "FiChevronLeft" },
  "DocumentNext": { "reactIcon": true, "tag": "FiChevronRight" }
}
```

Place this object under site. These controls use icons by default. The generated config may inherit reactIcon from icons.defaults; set an individual reactIcon to false to show text. Optional size, colorDark and colorLight fields customize icons. Ctrl+K (Cmd+K on macOS) and accessible names remain available. Previous and next document controls are configurable in the same way.

Use this PDF as a route

The example file lives at gitpagedocs/docs/versions/0.0.10/en/gitpagedocs-project-guide.pdf. The same English PDF can be linked from all three interface languages; localized PDFs may be supplied separately.

Site-wide defaults

```
"pdf": {
  "enabled": true,
  "previewEnabled": true,
  "downloadEnabled": true,
  "openInNewTabEnabled": true
}
```

Put these options under site in gitpagedocs/config.json. The PDF feature and its three actions are enabled by default. Individual PDF routes can override the preview and button settings.

Version configuration example

```
"routes-pdf": [{
  "id": 23,
  "title": { "en": "Project guide (PDF)" },
  "pdf": {
    "pathPdf": {
      "en": "gitpagedocs/docs/versions/0.0.10/en/gitpagedocs-project-guide.pdf"
    },
    "previewEnabled": true,
    "downloadEnabled": true,
    "openInNewTabEnabled": true
  }
}],
"menus-header-pdf": [{
  "id": 23,
  "en": { "title": "Project guide (PDF)", "path-click": "page:23" }
}]
```

Merge these collections into the version config; do not replace unrelated routes. Add localized title, menu and path entries as needed. The generated example already includes English, Portuguese and Spanish interface entries.

Preview, download and hosting

Preview fetches the file, validates its PDF signature and passes a PDF Blob to the native browser viewer. Download and Open in new tab reuse the validated bytes. All three actions need cross-origin permission (CORS) from external servers. Errors remain visible. Disable preview to defer fetching until Download or Open in new tab is clicked.

PDF files under gitpagedocs/ are staged as public assets with their relative paths preserved. External sources must use HTTP or HTTPS. New tabs receive application/pdf Blob URLs, so attachment headers cannot force a source download on localhost or GitHub Pages. The browser still controls its PDF viewer preferences. Allow popups for the site if opening a tab is blocked.

A focused reading experience

Keep reading while moving through the documentation

Focus mode displays Markdown in a reading dialog. Eight buttons provide first, previous, next and last navigation for both sections and documents. Single chevrons move by section; double chevrons move by document. Boundary buttons jump to the start or end of each group. The dialog stays open, and a new document begins at its first section.

```
"FocusMode": true,
"focusMode": {
  "copyEnabled": true,
  "downloadEnabled": true,
  "sectionNavigationEnabled": true,
  "documentNavigationEnabled": true
}
```

Place these settings under site. Copy and Download .md use the complete original Markdown, even when focus shows only one section. Turning section navigation off shows the whole document. Document navigation skips entries without usable Markdown in the active language and respects route access checks.

Native narration

```
"accessibility": {
  "enabled": true,
  "narration": {
    "enabled": true,
    "defaultRate": 1,
    "defaultVolume": 1
  }
},
"mediaExclusivePlayback": true
```

The reader uses the browser Speech Synthesis API. Open Accessibility and reading to choose a voice, change speed and volume, then start, pause, resume or stop. The native voice selector, sliders and actions adapt to narrow screens. Available voices depend on the browser and operating system. Narration never starts automatically. Preferences are shared between the normal page and focus mode while the page remains mounted.

One audible source at a time

With mediaExclusivePlayback enabled, starting narration pauses the existing audio or video source. Starting another audible source stops narration. Navigating to another document also stops narration. Set the option to false only when simultaneous playback is intentional. Muted video remains compatible with an audible source.

The narration feature reads Markdown text. PDF text-to-speech and PDF accessibility are provided by the browser viewer and the PDF document itself; the Markdown narration control does not extract PDF text.

CLI and publishing

Common commands

Command	Purpose
gitpagedocs	Create the configuration and documentation scaffold.
gitpagedocs --help	Inspect the supported commands and flags.
gitpagedocs --layoutconfig	Generate the optional layout configuration.
gitpagedocs docs	Update managed documentation regions.
gitpagedocs mcp start	Start the MCP server over stdio for a client.
gitpagedocs pages deploy	Configure Pages and perform the Git publication workflow.
gitpagedocs password	Configure the documentation access gate.

Use --owner and --repo to target the repository. Run commands from its root. The CLI manual documents command-specific options, provider configuration and publishing behavior. Review generated changes before publishing.

View a public repository

Commit the generated gitpagedocs folder. The official viewer can load a public repository at the following address:

```
https://vidigal-code.github.io/git-page-docs/YOUR_USER/YOUR_REPOSITORY/
```

Deploy your own GitHub Pages site

```
gh auth login
npx gitpagedocs pages deploy --owner YOUR_USER --repo YOUR_REPOSITORY
```

This workflow may generate files, create commits and push to the remote. The generated Actions workflow obtains the viewer runtime, combines it with the target documentation and exports a static Next.js site. The configured base path must match the repository and any custom documentation subpath.

Publish the project packages

In the GitPageDocs repository, a main-branch push triggers the existing npm workflow when the package version is not yet published. It publishes the tools, MCP and CLI packages in dependency order using the repository secret. The Pages workflow deploys the viewer separately. Confirm both workflow results and all package versions after a release.

Do not commit npm tokens, provider credentials or private keys. Use environment variables and repository secrets. The static site cannot protect a publicly hosted file merely by hiding its menu entry.

Connect the MCP server

The server exposes 20 tools and 7 resources through stdio. It is a child process of the MCP client, without an HTTP port or background daemon. Installing the CLI includes the server and its runtime.

Example client configuration

```
{
  "servers": {
    "gitpagedocs": {
      "type": "stdio",
      "command": "node",
      "args": [
        "${workspaceFolder}/node_modules/@gitpagedocs/cli/index.mjs",
        "mcp", "start"
      ],
      "cwd": "${workspaceFolder}"
    }
  }
}
```

This example uses the VS Code MCP configuration format. Adapt the wrapper to your client and keep the working directory at the project root. Test list_files and project://readme before configuring a provider.

Tools by purpose

Files	list_files, read_file, write_file, search_project
Provider setup	list_ai_providers, list_ai_models, configure_ai_provider, ask_ai
Document generation	generate_documentation, generate_api_docs, generate_database_docs, generate_readme, generate_architecture_docs
Analysis and releases	analyze_source_code, analyze_repository, analyze_project, generate_changelog, generate_release_notes
Maintenance	validate_docs, update_documentation

Resources and write behavior

Read project://structure, project://docs, project://config, project://repository, project://readme, project://ai/providers and project://ai/models. Generation tools return text. Only write_file, configure_ai_provider and update_documentation write files; the last updates a managed region.

Provider credentials come from the MCP process environment; the server does not automatically load .env or the CLI vault. Configure permissions in the client. Selected content is sent to the configured provider when generation or analysis is requested. See mcp/README.md for every argument, side effect and limit.

Maintain and troubleshoot

The validation loop

Use the repository harness and public interfaces to cover configuration, loaders, components and user flows. After changing default configuration or UI labels, regenerate the scaffold and synchronize the bundled example and baselines. Documentation pages are generated from cli/content/docs-en.mjs, docs-pt.mjs and docs-es.mjs.

```
pnpm run typecheck
pnpm run lint
pnpm exec vitest run
pnpm run smoke:all
pnpm run build
# Set PORT=3100 in the process environment before browser tests.
pnpm run test:e2e
```

Troubleshooting checklist

Missing page	Check the route ID, localized file paths, version registration, menu destination and access settings.
Wrong menu nesting	Check submenus and the active-language objects. Confirm nestedMenusEnabled is true.
Missing icon	Enable reactIcon explicitly and use a supported React Icons export name. Review size and theme colors.
No narration voices	Check browser support and installed operating-system voices. Voices may arrive after the page loads.
PDF cannot preview	Check browser PDF support, the file URL and CORS. Every action requires valid PDF bytes; opening a tab also requires popup permission.
PDF download blocked	Check source accessibility and CORS. Host the file with the site or use a server that allows access.
MCP fails to start	Check Node.js, the executable path, working directory and client stderr. stdout is reserved for JSON-RPC.

Further reading

Project repository and README
Published GitPageDocs viewer
Complete English Markdown manual
MCP reference: all tools and resources

The versioned manuals include configuration, functionalities, getting started, source browsing, security, CLI, MCP, navigation, focus, accessibility, PDF containers, themes, architecture, deployment, configuration recipes, migration, troubleshooting and contribution. Consult the installed version when a setting differs from this edition.